

ADVANCED MACHINE LEARNING · WORKSHEET 05

Batch Gradient Descent

for Multiple Linear Regression

Lecture 5

Name	Branch / Year	Date
_____	_____	_____

SECTION 1 Why OLS is not enough

▷ HOOK

In Lecture 4, we did something elegant. We set the gradient of the loss to zero and solved directly for θ . The result was exact. So why do we need another method?

The closed-form OLS solution is:

$$\theta^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

There is no trial and error, no epoch, no learning rate. However, there are two major limitations:

Limitation 1 - Computational cost. Since $\mathbf{X}$ has $m + 1$ columns (including the intercept), $\mathbf{X}^\top \mathbf{X}$ has dimensions $(m + 1) \times (m + 1)$. The computational cost of directly inverting this matrix is approximately:

$$O(m^3)$$

If the model has 5 features, inversion is trivial. If it has millions of features, the inverse becomes computationally impossible.

Limitation 2 - Invertibility and numerical stability. The formula works only when $\mathbf{X}^\top \mathbf{X}$ is invertible (i.e., its determinant is non-zero). This requires the columns of $\mathbf{X}$ to be linearly independent.

★ KEY INSIGHT

If two features are exact duplicates, or one feature is a linear combination of another, $\mathbf{X}^\top \mathbf{X}$ becomes **singular**, so the inverse does not exist.

We need a method that does not require explicitly inverting $\mathbf{X}^\top \mathbf{X}$. That method is **Batch Gradient Descent**.

SECTION 2 Introduction to Gradient Descent

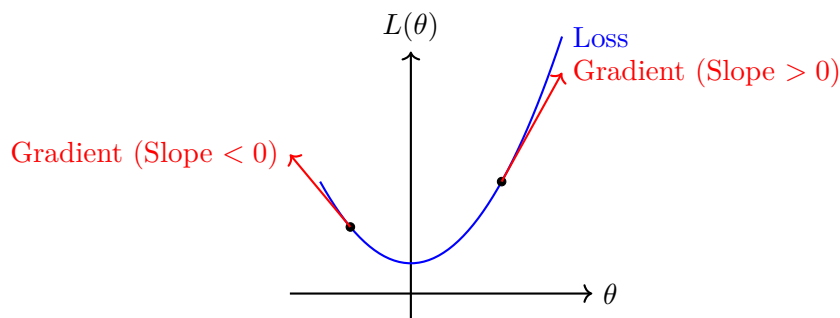
▷ HOOK

The objective of Gradient Descent is to reach the minimum of the loss function: the parameter values at the bottom give the smallest prediction error.

Before we talk about descent, let us make sure we remember what a gradient means from Calculus.
Single-variable case: For a function $f(x)$, the derivative $f'(x)$ tells us the slope.

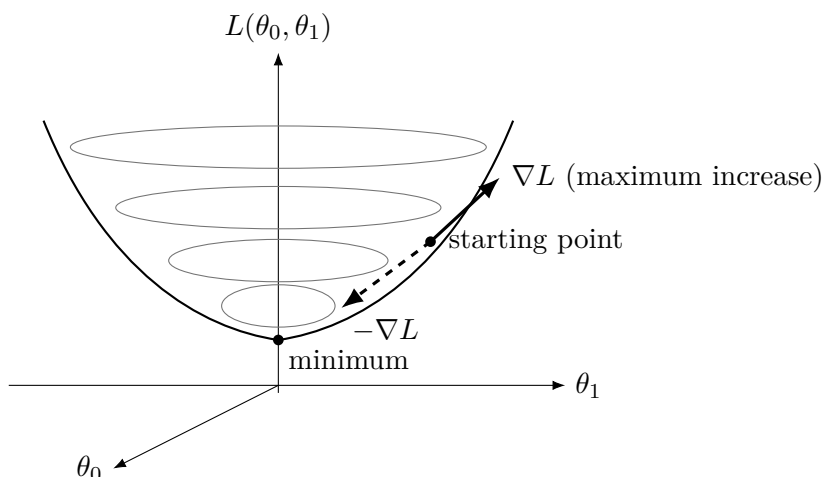
- If $f'(x) > 0$, the function increases as x increases.
- If $f'(x) < 0$, the function decreases as x increases.

If we want to *decrease* the function, we must move in the opposite direction of the derivative.



Multivariable case: For a function of several variables $f(\theta_0, \theta_1, \dots, \theta_m)$, the gradient is the vector of all partial derivatives:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial \theta_0} \\ \frac{\partial f}{\partial \theta_1} \\ \vdots \\ \frac{\partial f}{\partial \theta_m} \end{bmatrix}$$



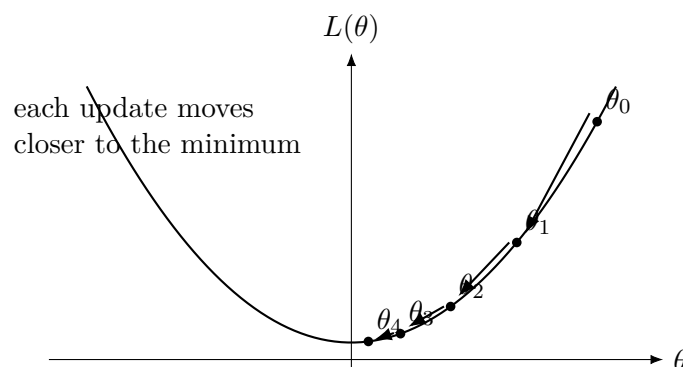
A convex bowl has one global minimum. From either side, the negative gradient points downhill toward the bottom.

◆ TAKEAWAY

The key geometric fact is: The gradient points in the direction of **steepest increase** of the function. Therefore, the **negative gradient** points in the direction of steepest **decrease**.

For a convex bowl-shaped loss, repeatedly following the negative gradient leads to the same global minimum from any starting point, provided the learning rate is appropriate.

Step-by-step parameter updates in one variable



Now introduce the update rule. For a single parameter, the value after one step is computed only from its old value:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \left. \frac{dL}{d\theta} \right|_{\theta_{\text{old}}}$$

Here, $\alpha > 0$ is the **learning rate**: a user-chosen hyperparameter controlling the size of each step. Gradient Descent applies this rule iteratively—old becomes new, the gradient is recomputed, and the process continues until convergence (when the loss or parameters change negligibly).

Let us generalize from one parameter to several parameters. For Simple Linear Regression, the loss depends on two parameters, $L(\theta_0, \theta_1)$; for Multiple Linear Regression it depends on the full parameter vector.

Generalizing to Multiple Linear Regression For MLR with m features, the model has $m + 1$ parameters including the intercept: $\boldsymbol{\theta} = [\theta_0, \theta_1, \dots, \theta_m]^\top$.

The coordinate-wise update is:

$$\theta_{j,\text{new}} = \theta_{j,\text{old}} - \alpha \left. \frac{\partial L}{\partial \theta_j} \right|_{\boldsymbol{\theta}_{\text{old}}} \quad \text{for } j = 0, 1, \dots, m$$

⚠ WARNING SIMULTANEOUS UPDATE

All partial derivatives must be computed using the **old** value of $\boldsymbol{\theta}$, and all parameters must be updated **simultaneously**. Do not update θ_0 , and then use the new θ_0 to compute the derivative for θ_1 .

Vector form Instead of writing $m + 1$ separate equations, we combine all updates into one vector equation:

$$\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta}_{\text{old}} - \alpha \nabla L(\boldsymbol{\theta}_{\text{old}})$$

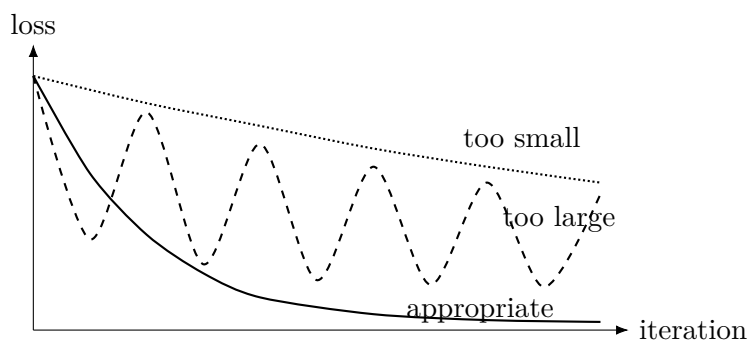
Breaking it down:

- $\boldsymbol{\theta}$ is the full parameter vector.
- ∇L is the vector of all partial derivatives.
- α is the learning rate.
- The minus sign appears because the gradient points uphill, but we want to move downhill.

Parameter initialization:

- For convex loss functions used in Linear Regression and Logistic Regression, initialize $\boldsymbol{\theta}_{\text{old}}$ as the zero vector.
- For non-convex Neural Network losses, parameters are initialized randomly. This will be covered in the parallel Deep Learning course.

Effect of the learning rate on convergence



PRACTICE P1 · SCENARIO EVALUATION

A data engineering team is deciding whether to use the closed-form OLS equation or Batch Gradient Descent for their regression models. Evaluate the following scenarios:

(a) **Scenario A:** The dataset has 10,000 rows but only 3 features (Age, Income, Credit Score).

- Should they use OLS or Gradient Descent? OLS
- Why? Only 3 features; the 4×4 inverse is cheap.

The number of features, not the 10,000 rows, controls the inverse size.

(b) **Scenario B:** A computer-vision regression model uses 50,000 image-derived features to predict manufacturing-defect severity.

- Should they use OLS or Gradient Descent? Gradient Descent
- Why? 50,000 features make $O(m^3)$ inversion impractical.

It avoids forming and inverting the enormous $50,001 \times 50,001$ matrix.

(c) **Scenario C:** A junior engineer accidentally includes 'Price_USD' and 'Price_Cents' as two separate input features (where 'Price_Cents' = $100 * \text{Price_USD}$).

- What specific error will the OLS formula throw? Singular matrix
- Can Batch Gradient Descent still run on this data? (Yes/No) Yes

Batch GD can run without the inverse, although the coefficients are not unique.

Homework PRACTICE P2 · DERIVE THE VECTORIZED UPDATE RULE

Fill in the missing components to create the full vector update rule.

1. Stack the old parameters into a vector θ :

$$\theta_{\text{old}} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_m \end{bmatrix}$$

2. Stack all the partial derivatives into the gradient vector ∇L :

$$\nabla L = \begin{bmatrix} \dots \partial L / \partial \theta_0 \dots \\ \dots \partial L / \partial \theta_1 \dots \\ \vdots \\ \dots \partial L / \partial \theta_m \dots \end{bmatrix}$$

3. Write the final simultaneous vector update equation.

$$\theta_{\text{new}} = \dots \theta_{\text{old}} - \alpha \nabla L(\theta_{\text{old}}) \dots$$

Coordinate form: $\theta_{j,\text{new}} = \theta_{j,\text{old}} - \alpha[\partial L/\partial \theta_j]$ at θ_{old} , $j = 0, \dots, m$.

Compute every derivative at θ_{old} , then update all parameters simultaneously.

SECTION 3 Deriving the Batch Gradient Descent update for MSE

▷ HOOK

Now we make the loss definition, gradient, and update rule mathematically consistent for the Mean Squared Error (MSE).

Use the loss from **Worksheet 4** Mean Squared Error is:

$$L(\theta) = \frac{1}{m} (\text{_____ } \mathbf{X}\theta - \mathbf{y} \text{ _____})^\top (\text{_____ } \mathbf{X}\theta - \mathbf{y} \text{ _____})$$

Equivalently:

$$L(\theta) = \frac{1}{m} \|\text{_____ } \mathbf{X}\theta - \mathbf{y} \text{ _____}\|_2^2$$

PRACTICE P4 · COMPUTE THE GRADIENT YOURSELF

Using the matrix calculus identities from Lecture 4, compute the gradient ∇L step-by-step.

Step 1: Expand the loss function.

$$L(\theta) = \frac{1}{m} [(\theta^\top \mathbf{X}^\top - \mathbf{y}^\top)(\mathbf{X}\theta - \mathbf{y})]$$

$$L(\theta) = \frac{1}{m} [\text{..... } \mathbf{y}^\top \mathbf{X}^\top \mathbf{X} \theta \text{} - 2\mathbf{y}^\top \mathbf{X}\theta + \text{..... } \theta^\top \mathbf{X}^\top \mathbf{X} \theta \text{}]$$

Step 2: Differentiate with respect to θ . Remember: $\frac{\partial}{\partial \theta}(\mathbf{y}^\top \mathbf{y}) = \mathbf{0}$, $\frac{\partial}{\partial \theta}(-2\mathbf{y}^\top \mathbf{X}\theta) = -2\mathbf{X}^\top \mathbf{y}$, and $\frac{\partial}{\partial \theta}(\theta^\top \mathbf{X}^\top \mathbf{X} \theta) = 2\mathbf{X}^\top \mathbf{X} \theta$.

$$\nabla L = \frac{1}{m} [\mathbf{0} - \text{..... } 2\mathbf{X}^\top \mathbf{y} \text{} + \text{..... } 2\mathbf{X}^\top \mathbf{X} \theta \text{}]$$

Step 3: Factor out the 2 and the $\mathbf{X}^\top$ to get the final gradient.

$$\nabla L = \frac{2}{m} \mathbf{X}^\top (\text{..... } \mathbf{X}\theta - \mathbf{y} \text{})$$

Step 4: Write the final Batch Gradient Descent Update Rule by plugging the gradient into the equation from P3.

$$\theta := \theta - \alpha \frac{2}{m} \text{..... } \mathbf{X}^\top (\mathbf{X}\theta - \mathbf{y}) \text{}$$

★ KEY INSIGHT

Why is this called **Batch** Gradient Descent? Because the gradient uses the full matrix $\mathbf{X}$ and the full target vector $\mathbf{y}$. Every training example contributes to every single update step.

Instructor note: Some books define the loss as:

$$L(\theta) = \frac{1}{2m} \|\mathbf{X}\theta - \mathbf{y}\|_2^2$$

With that convention, the factor 2 disappears from the gradient. In this lecture, we use the actual MSE definition, so the factor 2 remains.

SECTION 4 Worked example on the NST student dataset

We are predicting exam score using two features: hours studied and attendance percentage. The design matrix includes a column of ones for the intercept.

Student	x_1 (Hours)	x_2 (Attendance %)	y (Score)
1	2	60	35
2	4	80	55
3	6	90	70

Here, $m = 3$ (training examples). Initial parameter vector: $\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$. Learning rate: $\alpha = 0.0001$.

Step 1 - Design matrix and target vector

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 60 \\ 1 & 4 & 80 \\ 1 & 6 & 90 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 35 \\ 55 \\ 70 \end{bmatrix}$$

Step 2 - Compute predictions Since θ is the zero vector, $\hat{\mathbf{y}} = \mathbf{X}\theta = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$.

Step 3 - Compute residual vector $\mathbf{X}\theta - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 35 \\ 55 \\ 70 \end{bmatrix} = \begin{bmatrix} -35 \\ -55 \\ -70 \end{bmatrix}$.

Step 4 - Compute the matrix product $\mathbf{X}^\top(\mathbf{X}\theta - \mathbf{y})$

$$\mathbf{X}^\top(\mathbf{X}\theta - \mathbf{y}) = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 4 & 6 \\ 60 & 80 & 90 \end{bmatrix} \begin{bmatrix} -35 \\ -55 \\ -70 \end{bmatrix} = \begin{bmatrix} -160 \\ -710 \\ -12800 \end{bmatrix}$$

Step 5 - Compute the MSE gradient

$$\nabla L = \frac{2}{3} \begin{bmatrix} -160 \\ -710 \\ -12800 \end{bmatrix} = \begin{bmatrix} -106.67 \\ -473.33 \\ -8533.33 \end{bmatrix}$$

Step 6 - Update all parameters simultaneously

$$\theta_{\text{new}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0.0001 \begin{bmatrix} -106.67 \\ -473.33 \\ -8533.33 \end{bmatrix} = \begin{bmatrix} 0.011 \\ 0.047 \\ 0.853 \end{bmatrix}$$

PRACTICE P4 · COMPLETE EPOCH 2 BY HAND

Using the new parameter vector from Epoch 1 ($\theta = [0.011, 0.047, 0.853]^\top$), compute the next update. (Use a calculator!)

1. Compute new predictions $\hat{\mathbf{y}} = \mathbf{X}\theta$:

$$\hat{y}_1 = 1(0.011) + 2(0.047) + 60(0.853) = \dots\dots 51.285 \dots\dots$$

$$\hat{y}_2 = 1(0.011) + 4(0.047) + 80(0.853) = \dots\dots 68.439 \dots\dots$$

$$\hat{y}_3 = 1(0.011) + 6(0.047) + 90(0.853) = \dots\dots 77.063 \dots\dots$$

2. Compute the residual vector ($\hat{\mathbf{y}} - \mathbf{y}$):

$$\text{Residuals} = \begin{bmatrix} \dots 51.285 \dots - 35 \\ \dots 68.439 \dots - 55 \\ \dots 77.063 \dots - 70 \end{bmatrix} = \begin{bmatrix} \dots 16.285 \dots \\ \dots 13.439 \dots \\ \dots 7.063 \dots \end{bmatrix}$$

3. Explain briefly why the update for θ_2 (attendance) was so much larger than θ_1 (hours) in Epoch 1? What data preprocessing step could fix this imbalance?

Attendance values (60-90) are much larger than hours values (2-6),

so attendance contributes much larger terms to the gradient.

Standardize or normalize the features before training.

★ KEY INSIGHT

Feature scaling is essential before applying Gradient Descent. Putting features on comparable scales prevents a large-magnitude feature from dominating the gradient and usually makes convergence faster and more stable.

SECTION 5 Epochs and the full training loop

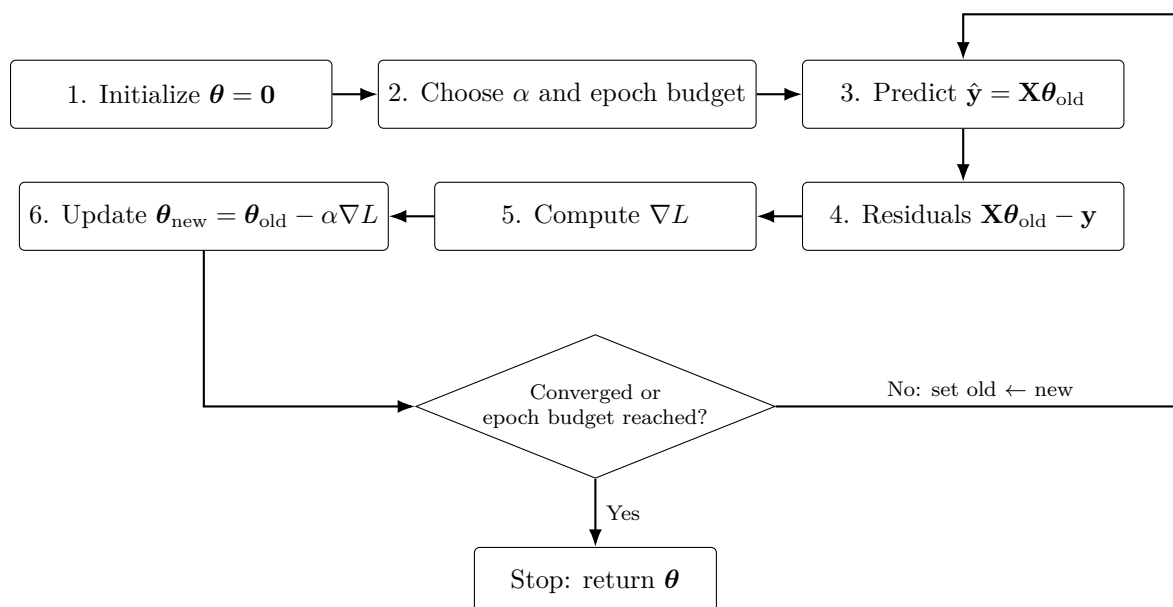
▷ HOOK

Everything we just did was one update. But training a model means repeating that update many times. Let us name the process carefully.

Training loop

1. **Initialize parameters.** For this convex Linear Regression loss, set $\boldsymbol{\theta} = \mathbf{0}$. Neural Networks use random initialization, as discussed in the parallel Deep Learning course.
2. **Choose the number of epochs.** In Batch Gradient Descent, one epoch means one complete pass through the full training dataset and one full-dataset gradient update.
3. **Compute predictions.** $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$
4. **Compute residuals.** $\mathbf{X}\boldsymbol{\theta} - \mathbf{y}$
5. **Compute the gradient.** $\nabla L = \frac{2}{m} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$
6. **Update parameters.** $\boldsymbol{\theta} := \boldsymbol{\theta} - \alpha \nabla L$
7. **Repeat** for the chosen number of epochs.

Training-loop flow diagram



Each “No” answer starts the next iteration using the parameter vector produced by the previous update.

PRACTICE P5 · EPOCH TRIVIA

If the NST dataset has 3 students and we train for 100 epochs using Batch Gradient Descent, how many times does the model use the full dataset to compute a gradient?

100 times - once per epoch.

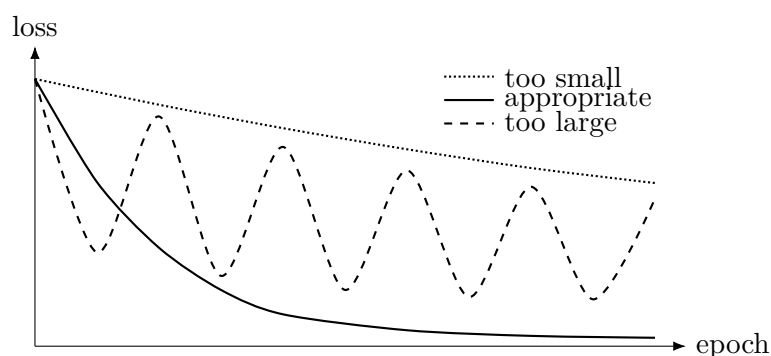
◆ TAKEAWAY

Gradient Descent is a general optimization algorithm, not an algorithm limited to Linear Regression. It can optimize many differentiable loss functions by repeatedly moving parameters in the negative-gradient direction.

SECTION 6 Learning rate, hyperparameter tuning, and trade-offs

The learning rate α is a **hyperparameter**. It is chosen by us before training, not learned by gradient descent. It controls how large each step is.

- **Case 1: α too small.** The loss decreases, but very slowly. The model is moving in the right direction, but each step is tiny.
- **Case 2: α too large.** The loss may oscillate or diverge. The direction may be downhill locally, but the step is so large that the algorithm overshoots the minimum.
- **Case 3: α well chosen.** The loss drops quickly at first and then gradually flattens near the minimum.



One graph, three learning-rate behaviours.

Hyperparameter tuning: To find the best α , we pick candidate values (e.g., 0.1, 0.01, 0.001), train the model separately with each value, and track the loss on a held-out **validation set**. (The test set should be kept for final evaluation, not for repeatedly selecting settings).

What is validation loss? Validation loss is the average prediction error on examples that are not used to update θ . For a validation set containing m_{val} examples,

$$L_{\text{val}} = \frac{1}{m_{\text{val}}} \sum_{i=1}^{m_{\text{val}}} (y_i - \hat{y}_i)^2.$$

After training with one candidate α , use the learned parameters to predict every validation example, square each residual, and average the squared residuals. Repeat this calculation for every candidate learning rate.

Step-by-step tuning process

1. Split the available development data into training and validation sets; keep the test set untouched.
2. Choose candidate learning rates on a logarithmic scale, such as 0.1, 0.01, 0.001.
3. Start each candidate run from the same zero vector and train for the same epoch budget.
4. Record both training loss and validation loss after every epoch.
5. Reject candidates that diverge or oscillate severely.
6. Compare the best validation loss reached by each remaining candidate.
7. Select the learning rate with the lowest validation loss because it has produced the smallest error on unseen validation data, then evaluate the final chosen setup once on the test set.

GUIDED DRY RUN: CALCULATE AND COMPARE VALIDATION LOSS

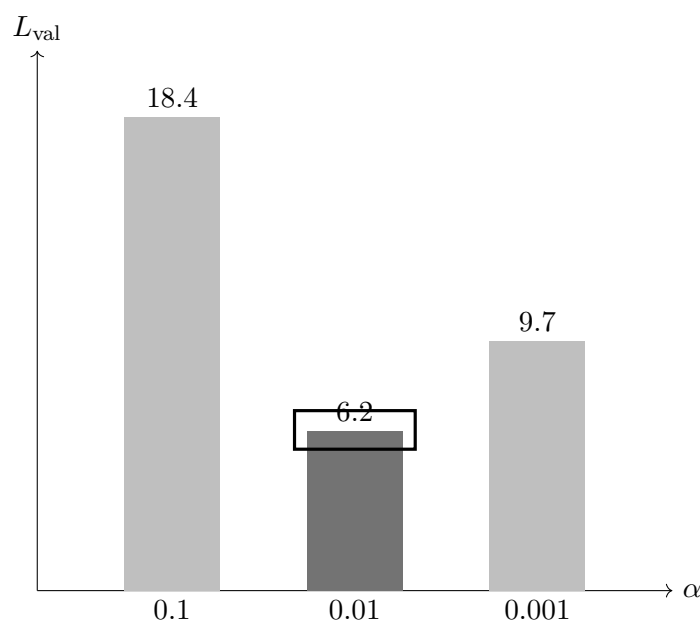
Suppose three validation residuals are 2, -1, 3. Calculate

$$L_{\text{val}} = \frac{2^2 + (-1)^2 + 3^2}{3} = \underline{14/3 \approx 4.67}.$$

Three separate training runs then give:

α	0.1	0.01	0.001
validation loss	18.4	6.2	9.7

Select $\alpha = \underline{0.01}$ because it has the lowest validation loss, 6.2.



The lowest validation-loss bar identifies the selected learning rate.

Advantages of Batch Gradient Descent

- It uses the full dataset for each gradient computation, so the update direction is stable.
- For a convex loss like MSE in linear regression, it can converge to a global minimum when the learning rate is chosen appropriately.
- It avoids explicitly computing $(\mathbf{X}^\top \mathbf{X})^{-1}$.
- It can still run when the inverse-based closed-form formula fails because $\mathbf{X}^\top \mathbf{X}$ is singular.

Disadvantages of Batch Gradient Descent

- Every epoch requires a full pass over the entire dataset.
- It can be slow when the dataset is very large.
- The whole dataset must usually be available to compute the full batch gradient efficiently.
- Poor learning-rate choices can make training extremely slow or unstable.

PRACTICE P6 · TRUE OR FALSE WITH JUSTIFICATION

For each statement write **T** or **F**. Where it says “Justify”, add a one-line explanation.

- (a) If the gradient points downhill, the loss is guaranteed to decrease after the parameter update, regardless of the learning rate. [**F**]
 Justify: A too-large learning rate can overshoot and increase or diverge the loss.
- (b) We use the test set to choose the best learning rate α . [**F**]
 Justify: Use validation data for tuning; keep the test set for final evaluation.
- (c) Batch Gradient Descent explicitly computes the inverse of $\mathbf{X}^\top \mathbf{X}$. [**F**]
- (d) Because Batch GD uses all training examples in every update, it is well-suited for a dataset with 50 million rows. [**F**]
 Justify: Each update scans all 50 million rows, so mini-batch GD is preferable.

BY THE END OF THIS SHEET, YOU CAN

- ☐ Explain why OLS is computationally expensive for large features and fails for collinear data.
- ☐ Define the gradient and explain its geometric meaning (direction of steepest increase).
- ☐ Complete the mathematical derivation for the Batch Gradient Descent update rule using the MSE loss function.

- ☐ Execute one full epoch of Batch Gradient Descent by hand using vectorized matrix operations.
- ☐ Explain the role of the learning rate α and the effects of it being too high or too low.
- ☐ Sequence the full training loop of Batch Gradient Descent.